

ContentAgent

Documentación del Esquema de Base de Datos

Este documento describe el modelo relacional completo de la plataforma ContentAgent. Incluye el diagrama entidad-relación, la descripción de cada tabla, sus campos, tipos de dato, restricciones y el lugar del código donde se utilizan.

Tablas físicas	17
Vistas (VIEW)	1
Campos totales	149
Relaciones FK	19
Grupos temáticos	7
Motor BD	PostgreSQL (Supabase)

Grupos Temáticos

Grupo	Tablas	Descripción
Auth (Supabase)	auth.users	Sistema de autenticación gestionado por Supabase. Tabla externa.
Noticias & Tendencias	news_items, news_interactions, user_opinions	Almacenamiento y ranking personalizado de noticias via Google News RSS.
Contenido Generado	contents	Todo el contenido generado por IA: shorts, carruseles, posts y threads.
Gestión de Equipos	team_members, team_invitations	Colaboración: invitaciones y miembros de equipo con roles.
Roles & Seguridad	user_roles, user_bans, admin_audit_logs	Control de acceso, bans y auditoría de acciones administrativas.
IA & Cuotas	ai_usage_logs, user_quotas	Registro de consumo de IA y límites personalizados por usuario.
Facturación	plans, credit_packs, subscriptions, payments, credit_transactions	Planes, créditos, suscripciones y pagos.
Configuración	app_settings	Parámetros globales de la aplicación editables por el admin.

Diagrama Entidad-Relación

Representación visual del modelo de datos. Las líneas representan relaciones de clave foránea (FK). Los colores identifican el grupo temático de cada tabla.

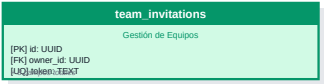
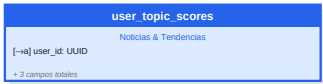
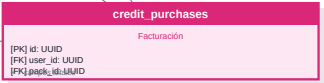
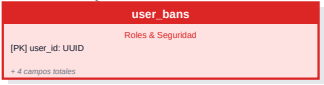
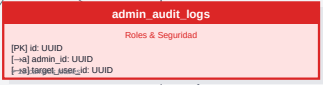
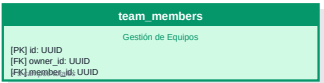


Diagrama Entidad-Relación – ContentAgent DB



- Grupos:
- Auth (Supabase)
 - Noticias & Tendencias
 - Contenido Generado
 - Gestión de Equipos
 - Roles & Seguridad
 - IA & Cuotas
 - Facturación
 - Configuración

Relaciones (Claves Foráneas)

Tabla origen	Tabla destino	Descripción de la FK
news_items	auth.users	news_items.user_id → auth.users.id
news_interactions	auth.users	news_interactions.user_id → auth.users.id
news_interactions	news_items	news_interactions.news_item_id → news_items.id
contents	auth.users	contents.user_id → auth.users.id
contents	news_items	contents.source_news_id → news_items.id
team_members	auth.users	team_members.owner_id / member_id → auth.users.id
team_invitations	auth.users	team_invitations.owner_id → auth.users.id
user_roles	auth.users	user_roles.user_id → auth.users.id
user_bans	auth.users	user_bans.user_id → auth.users.id
ai_usage_logs	auth.users	ai_usage_logs.user_id → auth.users.id
user_quotas	auth.users	user_quotas.user_id → auth.users.id
subscriptions	auth.users	subscriptions.user_id → auth.users.id
subscriptions	plans	subscriptions.plan_id → plans.id
payments	auth.users	payments.user_id → auth.users.id
payments	subscriptions	payments.subscription_id → subscriptions.id
credit_purchases	auth.users	credit_purchases.user_id → auth.users.id
credit_purchases	credit_packs	credit_purchases.pack_id → credit_packs.id
credit_purchases	payments	credit_purchases.payment_id → payments.id
admin_audit_logs	auth.users	admin_audit_logs.admin_id → auth.users.id

Documentación Detallada por Tabla

Auth (Supabase)

auth.users [TABLE]

Tabla de autenticación gestionada por Supabase Auth. Almacena credenciales y metadatos de sesión de todos los usuarios registrados en la plataforma.

Usado en: Referenciada (FK) desde todas las tablas del dominio mediante el campo `user_id`.

Campo	Tipo	Restricción	Descripción
id	UUID	PK	Identificador único del usuario (auth token)
email	VARCHAR		Dirección de correo electrónico
created_at	TIMESTAMPTZ		Fecha de registro
last_sign_in	TIMESTAMPTZ		Último inicio de sesión

Noticias & Tendencias

news_items [TABLE]

Caché de noticias obtenidas de Google News RSS. Cada usuario tiene sus propios registros (aislados por `user_id`). La deduplicación se hace por (`user_id`, `source_url`).

Usado en: Leída en `trends_page`, `rank-news API` y `generate-content`. Escrita en `fetch-news API`. Referenciada desde `contents (source_news_id)` y `news_interactions`.

Campo	Tipo	Restricción	Descripción
id	UUID	PK	Identificador único de la noticia
user_id	UUID	FK→auth.users	Propietario (usuario que buscó la noticia)
workspace_id	UUID		Workspace opcional (multi-tenant)
title	TEXT		Titular de la noticia (mejorado por IA)
summary	TEXT		Resumen en una frase generado por IA
source_url	TEXT	UQ	URL de la noticia original
source_name	TEXT		Nombre del medio de comunicación
published_at	TIMESTAMPTZ		Fecha de publicación original
topic	TEXT		Categoría: tecnología, ia, marketing...
language	TEXT		Idioma: 'es' o 'en' (default: es)
raw	JSONB		Datos completos del RSS en crudo
fetched_at	TIMESTAMPTZ		Fecha en que se capturó la noticia
created_at	TIMESTAMPTZ		Fecha de inserción en BD

news_interactions [TABLE]

Registro de interacciones del usuario con cada noticia. Una interacción actualiza dinámicamente las puntuaciones de temas (`user_topic_scores VIEW`) para personalizar el feed.

Usado en: Escrita en `news-interaction API`. Leída en `analytics API` y `rank-news API` para calcular puntuaciones de `topics`.

Campo	Tipo	Restricción	Descripción
-------	------	-------------	-------------

id	UUID	PK	Identificador único
user_id	UUID	FK→auth.users	Usuario que interactuó
news_item_id	UUID	FK→news_items	Noticia con la que interactuó
interaction	ENUM		Tipo: saved dismissed used
created_at	TIMESTAMPZ		Fecha de la interacción

user_topic_scores [VIEW]

Vista calculada (VIEW, no tabla física). Agrega puntuaciones por topic para personalizar el ranking de noticias. Fórmula: saved=+2, used=+3, dismissed=-1.

Usado en: Leída en rank-news API para ordenar noticias y construir el boost_prompt enviado a la IA.

Campo	Tipo	Restricción	Descripción
user_id	UUID	→auth.users	Usuario al que pertenece la puntuación
topic	TEXT		Categoría temática
score	NUMERIC		Puntuación acumulada (mayor = más interés)

■■ Contenido Generado ■■

contents [TABLE]

Almacena todo el contenido generado por la IA para cada usuario. Soporta 4 formatos: short, carousel, post, thread. El campo metadata guarda la estructura enriquecida (JSON con beats, slides, tweets...).

Usado en: Escrita en generate-content y save-content API. Leída en library_page, analytics API, dashboard. El admin puede ver todos los contenidos.

Campo	Tipo	Restricción	Descripción
id	UUID	PK	Identificador único del contenido
user_id	UUID	FK→auth.users	Creador del contenido
title	TEXT		Título descriptivo dado por el usuario
body	TEXT		Texto plano del contenido generado
format	TEXT		Formato: short carousel post thread
topic	TEXT		Tema sobre el que trata
tone	TEXT		Tono: profesional, educativo, coloquial...
status	ENUM		Estado: draft published discarded
source_news_id	UUID	FK→news_items	Noticia de origen (opcional)
metadata	JSONB		Estructura IA: beats, slides, tweets...
created_at	TIMESTAMPZ		Fecha de creación
updated_at	TIMESTAMPZ		Última modificación (auto-actualizado)

■■ Gestión de Equipos ■■

team_members [TABLE]

Miembros reales de un equipo. El owner_id es el usuario que creó el equipo. member_id es el usuario invitado. El rol define permisos dentro del equipo.

Usado en: CRUD completo en team API. Consultada en views de equipo para mostrar colaboradores.

Campo	Tipo	Restricción	Descripción
id	UUID	PK	Identificador único del registro
owner_id	UUID	FK→auth.users	Propietario del equipo
member_id	UUID	FK→auth.users	Usuario miembro
role	ENUM		Rol: owner admin editor viewer
created_at	TIMESTAMPTZ		Fecha de incorporación al equipo

team_invitations [TABLE]

Invitaciones pendientes para unirse a un equipo. El token es único por invitación. Las invitaciones expiran en 7 días. Un mismo email solo puede tener una invitación pending.

Usado en: Creada en invite-member API. Consultada y actualizada en team API. El usuario invitado la acepta mediante un link.

Campo	Tipo	Restricción	Descripción
id	UUID	PK	Identificador único
owner_id	UUID	FK→auth.users	Quien envía la invitación
email	TEXT		Email del invitado
role	ENUM		Rol asignado al aceptar: editor (default)
status	ENUM		Estado: pending accepted revoked expired
token	TEXT	UQ	Token hexadecimal de verificación
created_at	TIMESTAMPTZ		Fecha de creación
expires_at	TIMESTAMPTZ		Expiración (created_at + 7 días)

■■ Roles & Seguridad ■■

user_roles [TABLE]

Define el rol global de cada usuario en la plataforma (admin o user). Se consulta en todas las verificaciones de administrador. La función has_role() es SECURITY DEFINER para evitar recursión en RLS.

Usado en: Consultada en _require_admin() de admin API, _is_admin() de views, overview API.

Campo	Tipo	Restricción	Descripción
id	UUID	PK	Identificador único
user_id	UUID	FK→auth.users	Usuario al que se asigna el rol
role	ENUM		Rol: admin user
created_at	TIMESTAMPTZ		Fecha de asignación del rol

user_bans [TABLE]

Lista de usuarios baneados. Si el user_id aparece aquí, check_quota() bloquea todas las generaciones de IA. El admin puede banear/desbanear desde el panel.

Usado en: Consultada en check_quota() de quota_service antes de cada generación. Escrita desde ban-user y unban-user API.

Campo	Tipo	Restricción	Descripción
user_id	UUID	PK/FK→auth.users	Usuario baneado
reason	TEXT		Motivo del baneo

banned_by	UUID		Admin que aplicó el baneo
created_at	TIMESTAMP TZ		Fecha del baneo

admin_audit_logs [TABLE]

Registro inmutable de todas las acciones realizadas por administradores. Se inserta automáticamente con `_audit_log()` después de cada acción admin (ban, set-admin, set-quota, delete-content, etc.).

Usado en: Escrita en todas las acciones de admin API (`_audit_log`). Leída en audit-logs API del panel admin.

Campo	Tipo	Restricción	Descripción
id	UUID	PK	Identificador único
admin_id	UUID	→auth.users	Admin que realizó la acción
admin_email	TEXT		Email del admin (snapshot)
target_user_id	UUID	→auth.users	Usuario afectado
target_user_email	TEXT		Email del afectado (snapshot)
action	TEXT		Acción: ban_user, set_admin, delete_content...
payload	JSONB		Datos de entrada de la acción
result	JSONB		Resultado: success o error
success	BOOLEAN		Si la acción tuvo éxito
error_message	TEXT		Mensaje de error si falló
ip_address	TEXT		IP del administrador
user_agent	TEXT		Navegador/cliente del admin
created_at	TIMESTAMP TZ		Fecha de la acción

■ ■ IA & Cuotas ■ ■

ai_usage_logs [TABLE]

Log de todas las llamadas a la IA (Gemini/Lovable). Registra tokens consumidos, coste estimado y si fue exitosa. `check_quota()` cuenta las filas de hoy para verificar límites diarios.

Usado en: Escrita en `log_ai_usage()` de `quota_service` tras cada generación. Leída en `check_quota()`, `analytics API`, `ai-logs API` del admin.

Campo	Tipo	Restricción	Descripción
id	UUID	PK	Identificador único
user_id	UUID	FK→auth.users	Usuario que generó
function_name	TEXT		Función: generate-content, generate-insights
model	TEXT		Modelo IA: gemini-2.5-flash, gemini-2.5-pro
kind	TEXT		Tipo: content image
tokens_in	INTEGER		Tokens de entrada (prompt)
tokens_out	INTEGER		Tokens de salida (respuesta)
images_count	INTEGER		Imágenes generadas (si aplica)
estimated_cost_usd	NUMERIC		Coste estimado en USD
success	BOOLEAN		Si la llamada fue exitosa
error_message	TEXT		Mensaje de error si falló
metadata	JSONB		Contexto adicional: formato, topic...

created_at	TIMESTAMPZ		Fecha de la llamada
------------	------------	--	---------------------

user_quotas [TABLE]

Límites personalizados por usuario. Si no existe registro para un usuario se aplican los valores por defecto del código (50/día contenido, 20/día imagen). El admin puede ajustar estos límites.

Usado en: Leída en `check_quota()` de `quota_service`. Escrita en `set-quota` API del admin.

Campo	Tipo	Restricción	Descripción
user_id	UUID	PK/FK→auth.users	Usuario al que aplican las cuotas
daily_content_limit	INTEGER		Máximo de contenidos por día (default 50)
daily_image_limit	INTEGER		Máximo de imágenes por día (default 30)
monthly_content_limit	INTEGER		Máximo de contenidos por mes (default 1000)
monthly_image_limit	INTEGER		Máximo de imágenes por mes (default 500)
created_at	TIMESTAMPZ		Fecha de creación
updated_at	TIMESTAMPZ		Última actualización (auto-trigger)

■■ Facturación ■■

plans [TABLE]

Catálogo de planes de suscripción: Free, Pro, Business. Incluye precios mensuales y anuales, créditos mensuales de contenido e imagen, y features en JSON.

Usado en: Leída en página de precios/planes. Referenciada desde `subscriptions`.

Campo	Tipo	Restricción	Descripción
id	UUID	PK	Identificador único
key	TEXT	UQ	Clave: free, pro, business
name	TEXT		Nombre visible: Free, Pro, Business
description	TEXT		Descripción del plan
price_monthly_cents	INTEGER		Precio mensual en céntimos (0=gratis)
price_yearly_cents	INTEGER		Precio anual en céntimos
currency	TEXT		Moneda: EUR (default)
monthly_credits	INTEGER		Créditos de contenido por mes
monthly_image_credits	INTEGER		Créditos de imagen por mes
features	JSONB		Lista de características del plan
active	BOOLEAN		Si el plan está disponible
sort_order	INTEGER		Orden de visualización
created_at	TIMESTAMPZ		Fecha de creación
updated_at	TIMESTAMPZ		Última actualización

credit_packs [TABLE]

Paquetes de créditos adicionales que se pueden comprar puntualmente: Starter (100), Boost (500), Mega (2000). Cada pack tiene créditos de contenido e imagen.

Usado en: Referenciada desde `credit_purchases`. Leída en página de compra de créditos.

Campo	Tipo	Restricción	Descripción
id	UUID	PK	Identificador único
key	TEXT	UQ	Clave: starter, boost, mega
name	TEXT		Nombre visible
credits	INTEGER		Créditos de contenido incluidos
image_credits	INTEGER		Créditos de imagen incluidos
price_cents	INTEGER		Precio en céntimos de euro
currency	TEXT		Moneda: EUR
active	BOOLEAN		Si el pack está disponible
sort_order	INTEGER		Orden de visualización
created_at	TIMESTAMPZ		Fecha de creación

subscriptions [TABLE]

Suscripciones activas de usuarios a planes. Gestiona el ciclo de facturación (start/end), estado (trialing, active, canceled...) y el identificador externo del proveedor de pagos.

Usado en: Leída en perfil de usuario para mostrar plan activo. Referenciada desde payments.

Campo	Tipo	Restricción	Descripción
id	UUID	PK	Identificador único
user_id	UUID	FK→auth.users	Suscriptor
plan_id	UUID	FK→plans	Plan contratado
status	ENUM		Estado: trialing active canceled past_due paused
interval	ENUM		Ciclo: monthly yearly
current_period_start	TIMESTAMPZ		Inicio del período actual
current_period_end	TIMESTAMPZ		Fin del período actual
canceled_at	TIMESTAMPZ		Fecha de cancelación
trial_end	TIMESTAMPZ		Fin del período de prueba
external_id	TEXT		ID en el proveedor de pagos (Stripe, etc.)
created_at	TIMESTAMPZ		Fecha de creación
updated_at	TIMESTAMPZ		Última actualización

payments [TABLE]

Historial de pagos realizados. Puede ser por suscripción, compra de créditos adicionales o ajuste manual. El status refleja el resultado del procesador de pagos.

Usado en: Leída en historial de facturación del usuario. Referenciada desde credit_purchases.

Campo	Tipo	Restricción	Descripción
id	UUID	PK	Identificador único
user_id	UUID	FK→auth.users	Usuario que pagó
subscription_id	UUID	FK→subscriptions	Suscripción asociada (si aplica)
amount_cents	INTEGER		Importe en céntimos
currency	TEXT		Moneda: EUR
status	ENUM		Estado: succeeded pending failed refunded

reason	TEXT		Motivo: subscription credit_pack manual
description	TEXT		Descripción del cobro
external_id	TEXT		ID en el proveedor de pagos
metadata	JSONB		Datos adicionales del pago
created_at	TIMESTAMPTZ		Fecha del pago

credit_purchases [TABLE]

Registro de compras de paquetes de créditos adicionales. Vincula el pago con el pack adquirido y los créditos concedidos al usuario.

Usado en: Escrita al confirmar pago de un credit_pack. Leída para mostrar historial de compras.

Campo	Tipo	Restricción	Descripción
id	UUID	PK	Identificador único
user_id	UUID	FK→auth.users	Comprador
pack_id	UUID	FK→credit_packs	Pack comprado
payment_id	UUID	FK→payments	Pago asociado
credits_granted	INTEGER		Créditos de contenido concedidos
image_credits_granted	INTEGER		Créditos de imagen concedidos
created_at	TIMESTAMPTZ		Fecha de la compra

■■ Configuración ■■

app_settings [TABLE]

Configuración global de la aplicación almacenada como clave-valor JSON. Incluye modo mantenimiento, banner global y modelos de IA por defecto. Editable solo por admins.

Usado en: Leída en get-settings API del admin. Escrita en update-setting API.

Campo	Tipo	Restricción	Descripción
key	TEXT	PK	Clave: maintenance_mode, global_banner, default_models
value	JSONB		Valor en formato JSON
updated_at	TIMESTAMPTZ		Última actualización
updated_by	UUID		Admin que lo actualizó

Cómo Construir la BD en tier_database

1. Instalar PostgreSQL

Descarga e instala PostgreSQL 15+ desde <https://www.postgresql.org/download/>. En Windows puedes usar el instalador o via conda: `conda install -c conda-forge postgresql`

2. Instalar psycopg2

```
pip install psycopg2-binary
```

Este adaptador conecta Django con PostgreSQL.

3. Instalar django-extensions y pillow (opcional)

```
pip install django-extensions pillow
```

Permite generar diagramas del modelo con el comando `graph_models`.

4. Configurar la conexión en .env

Añade en el fichero `.env`:

```
DB_NAME=contentagent
```

```
DB_USER=postgres
```

```
DB_PASSWORD=tu_password
```

```
DB_HOST=localhost
```

```
DB_PORT=5432
```

5. Activar tier_database en Django

En `content_agent/settings.py` cambia `DATABASES` para usar PostgreSQL y añade 'tier_database' a `INSTALLED_APPS`.

6. Ejecutar migraciones

```
python manage.py makemigrations tier_database
```

```
python manage.py migrate
```

Esto creará todas las tablas en tu PostgreSQL local.

7. Alternativa: usar SQLite para desarrollo

Si no quieres instalar PostgreSQL, cambia `ENGINE` a `'django.db.backends.sqlite3'` y `NAME` a `BASE_DIR / 'db.sqlite3'`. SQLite no necesita instalación adicional.